

Data Skew Handling in Spark

Data Skew is a common issue in distributed systems like Apache Spark, and it can cause severe performance degradation or job failures due to one or more tasks taking much longer than others.

How to Identify Data Skew in PySpark

1. Task Duration Imbalance:

- Use the **Spark UI** (<http://<spark-driver>:4040>) → **Stages** tab.
- Look for stages where a few tasks take significantly longer than others.
- If most tasks finish in seconds but a few take minutes, that's a strong indicator of skew.

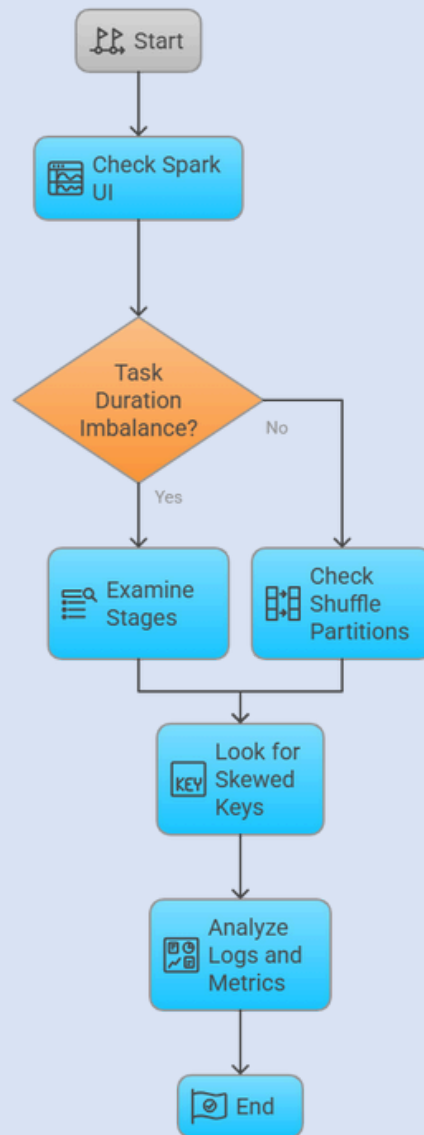
2. Skewed Shuffle Partitions:

- Check for stages involving wide transformations like `groupByKey()`, `join()`, or `reduceByKey()`.
- Under the "Tasks" tab, look at the input size and run time of each task.
- Skewed keys will show disproportionate data sizes for specific partitions.

3. Logs and Metrics:

- Enable detailed logging (especially for jobs involving joins or aggregations).
- Use `.rdd.getNumPartitions()` or `.rdd.glom().map(len).collect()` to inspect distribution manually.

Identifying Data Skew in PySpark



How to Resolve Data Skew in PySpark

1. Salting the Skewed Key

- Add a random number (salt) to the skewed key before the join or aggregation.
- Helps distribute the records more evenly.

```
from pyspark.sql.functions import col, concat_ws, rand

df_skewed = df.withColumn("salt", (rand() * 10).cast("int"))
df_skewed = df_skewed.withColumn("new_key", concat_ws("_", col("key"), col("salt")))
```

Data Skew Mitigation in PySpark



Identify Skewed Data

Recognizing data imbalance in the dataset



Add Salt Column

Introducing a random number to the data



Create New Key

Forming a new key by combining original key and salt



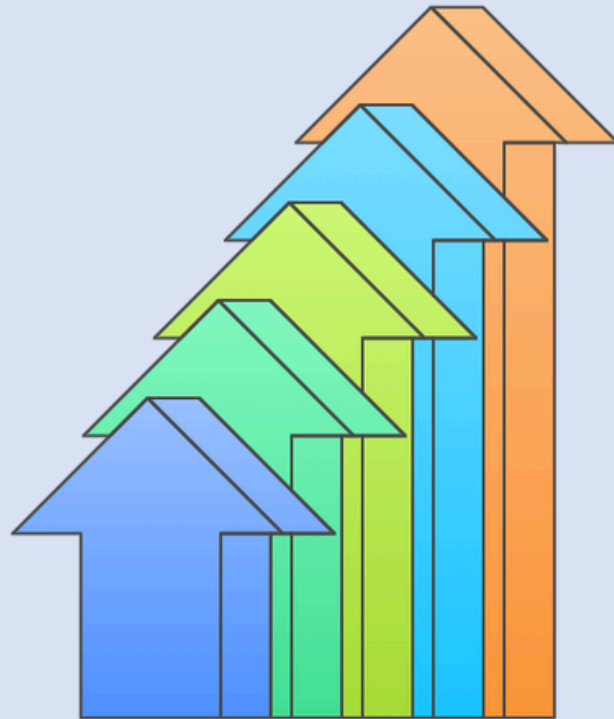
Perform Join/Aggregation

Executing data operations with the new key



Optional Desalting

Reverting the salt addition if needed

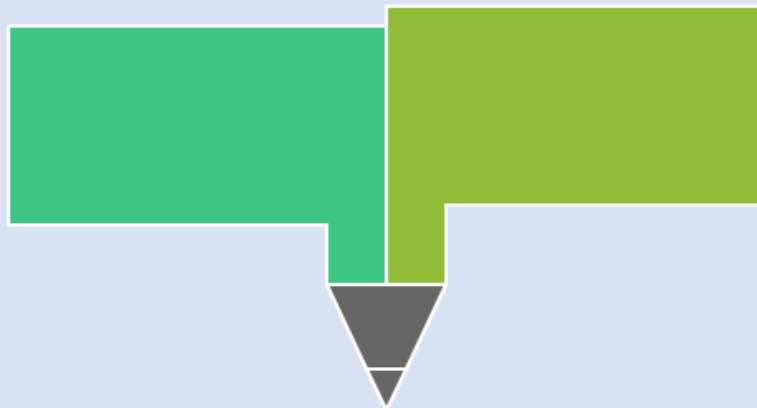


2. Broadcast Join (for small datasets)

- If one side of the join is small enough, broadcast it to avoid shuffling the large dataset.

```
from pyspark.sql.functions import broadcast

result = large_df.join(broadcast(small_df), "join_key")
```



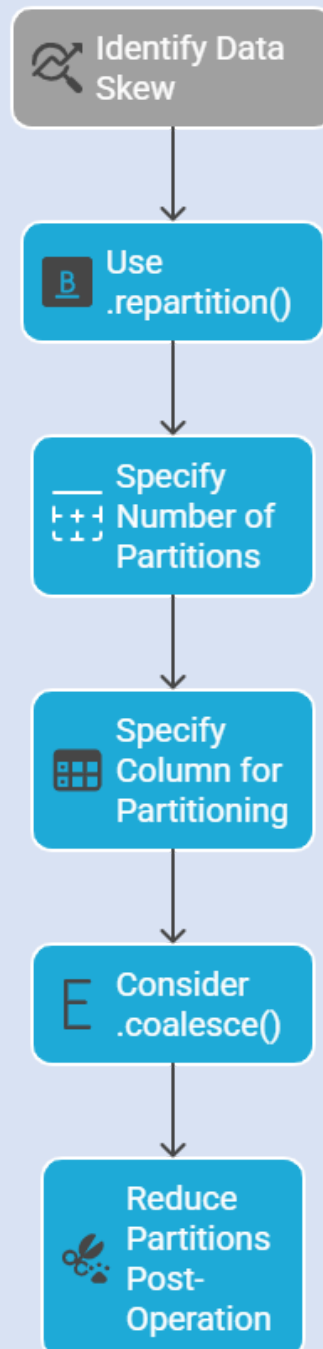
3. Repartitioning

- Manually repartition data using `.repartition()` or `.partitionBy()` to balance load.

```
df = df.repartition(100, "skewed_column")
```

- You can also coalesce after an expensive operation to reduce partitions.

Data Partitioning in Spark



4. Avoid `groupByKey()`

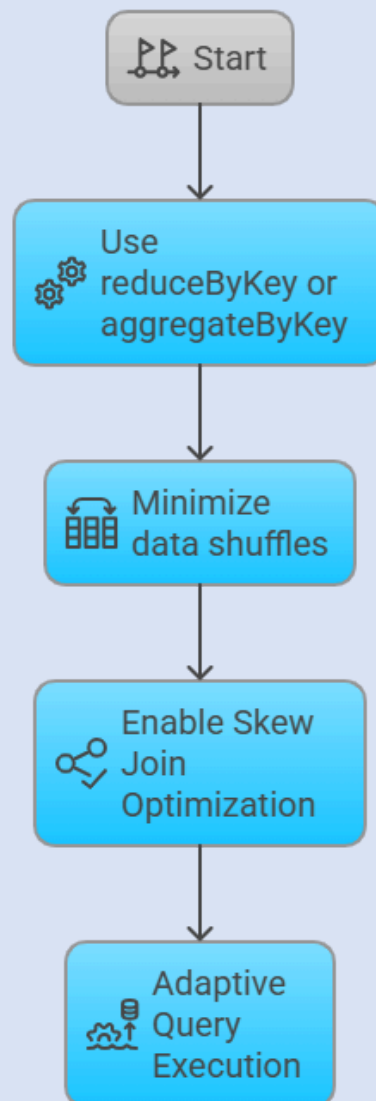
- Prefer `reduceByKey()` or `aggregateByKey()` which are more efficient and minimize shuffles.

5. Skew Join Optimization (in Spark 3+)

- If enabled, Spark can automatically handle skew joins using **adaptive query execution** (AQE).

```
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", True)
```

Spark Optimization Techniques

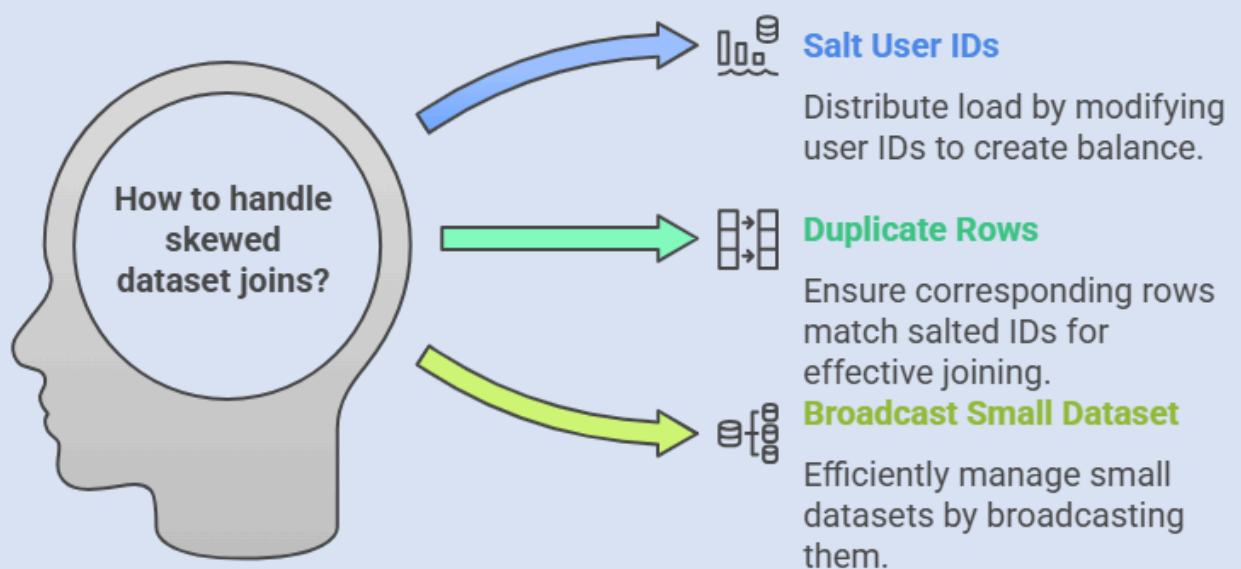


Example Scenario

Suppose your job is joining two large datasets on `user_id`, and `user_id = 123` has millions of rows while others have just a few. That **task processing `user_id = 123` will lag.**

Fix:

- **Salt** `user_id` in one dataset (e.g., append `_0` to `_9`) to split the skew.
- Duplicate corresponding rows in the other dataset with matching salts (or broadcast if it's small). Join on the salted keys.



Thank You
For downloading